NATIONAL UNIVERSITY OF MODERN LANGUAGES

Artificial Intelligence Lab Report No - 3

NAME: M Abdul Rafay

ROLL NO: FL23791

PROGRAM: BSSE (6th Semester)

COURSE: Artificial Intelligence

SUBMITTED TO: Mam Amna Sajjad

DATE: 22 Feb 2026 **DAY: Sunday**

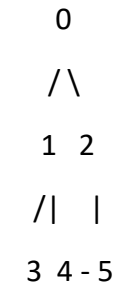
Implementation Of BFS and DFS Using An Example

1. Objective

The purpose of this lab is to implement and understand two fundamental graph traversal algorithms: Breadth-First Search (BFS) and Depth-First Search (DFS). Both algorithms are implemented in Python and tested on the same example graph. The output of each program is recorded below.

2. Example Graph

The following undirected graph with 6 nodes (0 to 5) and 7 edges is used for both algorithms. Both traversals start from Node 0.



Edges: 0-1, 0-2, 1-3, 1-4, 2-5, 3-4, 4-5

Adjacency List

Node	Neighbours
0	1, 2
1	0, 3, 4
2	0, 5
3	1, 4
4	1, 3, 5
5	2, 4

3. Breadth-First Search (BFS)

3.1 Concept

BFS visits nodes level by level using a Queue (First-In First-Out). It first visits all direct neighbours of the starting node, then their neighbours, and so on. It guarantees the shortest path in an unweighted graph.

3.2 Step-by-Step Trace (Start = Node 0)

Step	What Happens	Queue [front → back]	Visited
—	Start. Enqueue Node 0, mark visited.	[0]	{ 0 }
1	Dequeue 0. Neighbours 1, 2 are new → enqueue both.	[1, 2]	{ 0, 1, 2 }
2	Dequeue 1. Neighbours: 0(visited), 3, 4 → enqueue 3, 4.	[2, 3, 4]	{ 0, 1, 2, 3, 4 }
3	Dequeue 2. Neighbours: 0(visited), 5 → enqueue 5.	[3, 4, 5]	{ 0, 1, 2, 3, 4, 5 }
4	Dequeue 3. Neighbours: 1, 4 — both visited. Nothing added.	[4, 5]	{ 0, 1, 2, 3, 4, 5 }
5	Dequeue 4. Neighbours: 1, 3, 5 — all visited. Nothing added.	[5]	{ 0, 1, 2, 3, 4, 5 }
6	Dequeue 5. Neighbours: 2, 4 — visited. Queue empty. Done.	[empty]	{ 0, 1, 2, 3, 4, 5 }

BFS Result: 0 → 1 → 2 → 3 → 4 → 5

3.3 Python Code (BFS)

```
from collections import deque

# Graph represented as an adjacency list
graph = {
  0: [1, 2],
  1: [0, 3, 4],
  2: [0, 5],
  3: [1, 4],
```

```
    4: [1, 3, 5],
    5: [2, 4]
}

def bfs(graph, start):
    visited = []        # List to store visited order
    queue = deque()      # Queue for BFS
    queue.append(start)   # Start from the given node
    seen = set()         # To avoid revisiting
    seen.add(start)

    while queue:
        node = queue.popleft()   # Remove from front of queue
        visited.append(node)     # Mark as visited

        for neighbour in graph[node]:
            if neighbour not in seen:
                seen.add(neighbour)
                queue.append(neighbour)

    return visited

result = bfs(graph, 0)
print('BFS Traversal Order:', result)

# Show step-by-step
print()
print('Step-by-step BFS from Node 0:')
seen = set()
queue = deque([0])
seen.add(0)
step = 1
while queue:
    node = queue.popleft()
    print(f'  Step {step}: Visit Node {node}')
    step += 1
    for neighbour in graph[node]:
        if neighbour not in seen:
            seen.add(neighbour)
            queue.append(neighbour)
```

3.4 Output

```
...  BFS Traversal Order: [0, 1, 2, 3, 4, 5]

    Step-by-step BFS from Node 0:
      Step 1: Visit Node 0
      Step 2: Visit Node 1
      Step 3: Visit Node 2
      Step 4: Visit Node 3
      Step 5: Visit Node 4
      Step 6: Visit Node 5
```

4. Depth-First Search (DFS)

4.1 Concept

DFS visits nodes by going as deep as possible along one branch before backtracking. It uses a Stack (Last-In First-Out) or recursion. It does not guarantee the shortest path but is useful for exploring all paths.

4.2 Step-by-Step Trace (Start = Node 0)

Step	What Happens	Call Stack [top]	Visited
—	Start at Node 0. Mark visited.	dfs(0)	{ 0 }
1	At 0 — first unvisited neighbour is 1. Go deeper.	dfs(0)→dfs(1)	{ 0, 1 }
2	At 1 — first unvisited neighbour is 3. Go deeper.	…→dfs(3)	{ 0, 1, 3 }
3	At 3 — unvisited neighbour is 4. Go deeper.	…→dfs(4)	{ 0, 1, 3, 4 }
4	At 4 — unvisited neighbour is 5. Go deeper.	…→dfs(5)	{ 0, 1, 3, 4, 5 }
5	At 5 — unvisited neighbour is 2. Go deeper.	…→dfs(2)	{ 0, 1, 2, 3, 4, 5 }
6	At 2 — all neighbours visited. Backtrack all the way. Done.	empty	{ 0, 1, 2, 3, 4, 5 }

DFS Result: 0 → 1 → 3 → 4 → 5 → 2

4.3 Python Code (DFS)

```
# Graph represented as an adjacency list
graph = {
   0: [1, 2],
   1: [0, 3, 4],
   2: [0, 5],
   3: [1, 4],
   4: [1, 3, 5],
   5: [2, 4]
}

def dfs(graph, node, visited=None, order=None):
   if visited is None:
      visited = set()
   if order is None:
      order = []

   visited.add(node)      # Mark current node as visited
   order.append(node)     # Record visit order

   for neighbour in graph[node]:
      if neighbour not in visited:
         dfs(graph, neighbour, visited, order)   # Recurse deeper

   return order

result = dfs(graph, 0)
print('DFS Traversal Order:', result)

# Show step-by-step
print()
print('Step-by-step DFS from Node 0:')
visited = set()
order = []
step_counter = [1]

def dfs_verbose(graph, node, visited, step_counter):
   visited.add(node)
   print(f'  Step {step_counter[0]}: Visit Node {node}')
   step_counter[0] += 1
   for neighbour in graph[node]:
```

```
        if neighbour not in visited:
            dfs_verbose(graph, neighbour, visited, step_counter)

    dfs_verbose(graph, 0, visited, step_counter)
```

4.4 Output

```
...  DFS Traversal Order: [0, 1, 3, 4, 5, 2]

    Step-by-step DFS from Node 0:
      Step 1: Visit Node 0
      Step 2: Visit Node 1
      Step 3: Visit Node 3
      Step 4: Visit Node 4
      Step 5: Visit Node 5
      Step 6: Visit Node 2
```

5. Comparison

Feature	BFS	DFS
Approach	Level by level	Deep path first, then backtrack
Data Structure	Queue (FIFO)	Stack / Recursion (LIFO)
Result on Example	0, 1, 2, 3, 4, 5	0, 1, 3, 4, 5, 2
Shortest Path?	Yes (unweighted)	Not guaranteed
Time Complexity	O(V + E)	O(V + E)
Space Complexity	O(V)	O(V)
Best For	Shortest path, nearest node	Cycle detection, maze solving

6. Conclusion

This lab implemented BFS and DFS in Python and traced both algorithms manually on the same 6-node undirected graph. BFS produced the traversal order 0, 1, 2, 3, 4, 5 by processing nodes level by level using a queue. DFS produced the order 0, 1, 3, 4, 5, 2 by following one path as deep as possible before backtracking using recursion.

Both algorithms successfully visited all six nodes and all seven edges exactly once. BFS is preferred when the shortest path is needed, while DFS is preferred for exhaustive path exploration and cycle detection.